TRƯỜNG: **ĐH CT TP.HCM** KHOA: **CÔNG NGHỆ THÔNG TIN** BỘ MÔN: **KHDL** MH: **TH HỌC MÁY**	**BUỔI 4. PHÂN CỤM (HIERARCHICAL - KMEAN)**	

A. MỤC TIÊU

- Xác định được bài toán phân cụm dựa trên dữ liệu thu thập được.
- Sử dụng thuật toán phân cụm phù hợp với bộ dữ liệu cụ thể.
- Đánh giá thực trạng bộ dữ liệu.
- Tiền xử lý dữ liệu (nếu có).
- Cài đặt thuật toán huấn luyện mô hình phân cụm cho bộ dữ liệu cụ thể: chia dữ liệu, chọn các siêu tham số,…).
- Cách dùng phương pháp Elbow để xác định K.

B. DỤNG CỤ - THIẾT BỊ THÍ NGHIỆM CHO MỘT SV

STT	Chủng loại – Quy cách vật tư	Số lượng	Đơn vị	Ghi chú
1	Computer	1	1	

C. NỘI DUNG THỰC HÀNH

I. Tóm tắt lý thuyết

1. Sử dụng Google Colab để viết chương trình:

B1. Truy cập địa chỉ: https://colab.research.google.com/

B2. Vào file → new notebook



B3. Kết nối máy chủ bằng cách click chuột vào Connect như trên hình.

2. Hierarchical clustering

Một kỹ thuật phân cụm thuộc phương pháp học không giám sát (unsupervised) trong học máy, được sử dụng để phân cụm dữ liệu mà không cần chỉ định trước số cụm mong muốn.

Thay vì xác định các cụm từ trước, phương pháp phân cụm phân cấp xây dựng một cấu trúc phân cấp các cụm từ nhỏ đến lớn (hoặc ngược lại), cho phép hiểu cách các mẫu dữ liệu nhóm lại ở nhiều mức độ chi tiết khác nhau. Phân cụm phân cấp có thể được thể hiện dưới dạng một cây phân cấp (dendrogram) để trực quan hóa.

Có hai cách tiếp cận chính:

- Phân cụm phân cấp tăng dần (Agglomerative Hierarchical Clustering): Còn được gọi là bottom-up, mỗi điểm dữ liệu bắt đầu là một cụm riêng lẻ. Các cụm được hợp nhất lại thành cụm lớn hơn dựa trên khoảng cách giữa chúng cho đến khi tất cả dữ liệu nằm trong một cụm duy nhất.
- Phân cụm phân cấp suy giảm (Divisive Hierarchical Clustering): Còn được gọi là top-down, toàn bộ dữ liệu ban đầu là một cụm duy nhất. Cụm này dần dần bị chia nhỏ thành các cụm con cho đến khi mỗi điểm dữ liệu là một cụm riêng.

Phân cụm tăng dần thường phổ biến hơn trong thực tế vì nó dễ triển khai và trực quan hơn.

Các bước cơ bản thực hiện phân cụm tăng dần:

- Tính khoảng cách giữa các điểm dữ liệu: Các điểm có thể được biểu diễn trong không gian đặc trưng, và khoảng cách giữa các điểm có thể được đo bằng các thước đo như khoảng cách Euclidean, khoảng cách Manhattan, hoặc các khoảng cách tùy chỉnh khác.
- Tìm cặp cụm gần nhất và hợp nhất: Tính toán và chọn cặp cụm gần nhất để hợp nhất. Khoảng cách giữa các cụm có thể được đo bằng các phương pháp như:
    - Phương pháp liên kết đơn (Single linkage): Khoảng cách giữa hai cụm là khoảng cách nhỏ nhất giữa các điểm của hai cụm.
    - Phương pháp liên kết đầy đủ (Complete linkage): Khoảng cách giữa hai cụm là khoảng cách lớn nhất giữa các điểm của hai cụm.
    - Phương pháp liên kết trung bình (Average linkage): Khoảng cách giữa hai cụm là khoảng cách trung bình giữa tất cả các điểm của hai cụm.
    - Liên kết tâm cụm (Centroid linkage): Khoảng cách giữa hai cụm là khoảng cách giữa các tâm của hai cụm.
- Lặp lại cho đến khi hợp nhất tất cả các cụm: Tiếp tục tìm và hợp nhất các cụm gần nhất cho đến khi đạt đến số cụm mong muốn hoặc tất cả các điểm dữ liệu nằm trong một cụm duy nhất.

  Cấu trúc phân cụm phân cấp có thể được biểu diễn bằng một **cây dendrogram**. Mỗi nhánh của cây thể hiện một cụm dữ liệu, giúp quan sát mối quan hệ giữa các điểm dữ liệu và cụm của chúng ở nhiều mức độ phân cụm khác nhau.

  Ví dụ 1 minh họa cây phân cụm phân cấp:

```
# Khai báo thư viện

import numpy as np
```

```
import matplotlib.pyplot as plt

from scipy.cluster.hierarchy import dendrogram, linkage

from sklearn.datasets import make_blobs

# Tạo dữ liệu mẫu ngẫu nhiên (15 mẫu dữ liệu)

X, y = make_blobs(n_samples=15, centers=3, random_state=42)

# Thực hiện phân cụm phân cấp tăng dần "ward" là phương pháp liên kết tổng thể

Z = linkage(X, method='ward')

# Vẽ dendrogram

plt.figure(figsize=(10, 7))

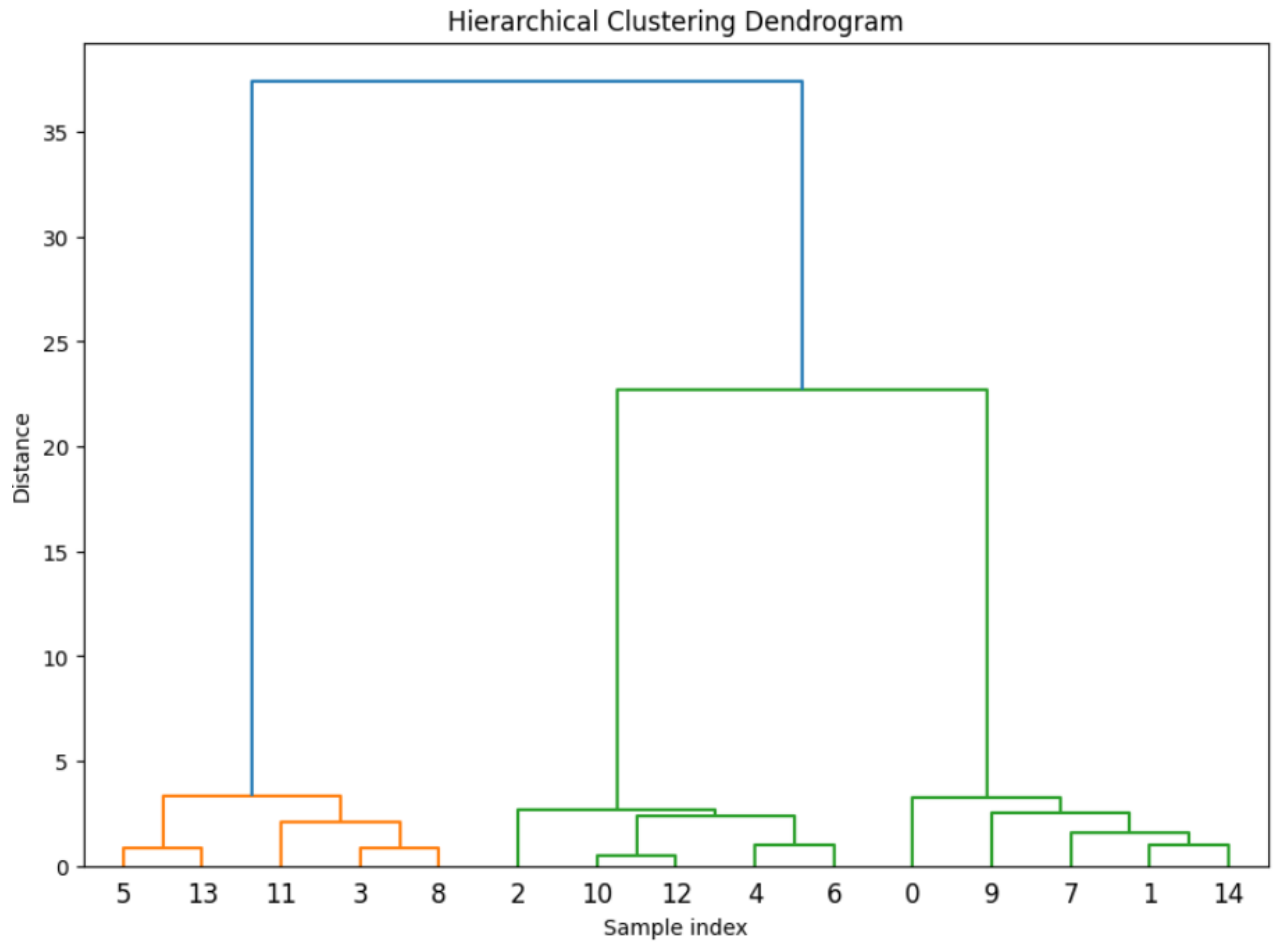
dendrogram(Z)

plt.title("Hierarchical Clustering Dendrogram")

plt.xlabel("Sample index")

plt.ylabel("Distance")

plt.show()
```



Ví dụ 2: cách thực hiện phân cụm phân cấp dựa trên hai thuộc tính lương và tuổi của người dùng.

Thư viện

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from scipy.cluster.hierarchy import dendrogram, linkage

from scipy.cluster.hierarchy import fcluster
```

Tạo tập dữ liệu mẫu (có thể đọc file dữ liệu dạng csv) giả định với 2 thuộc tính: Tuổi và Lương

```
np.random.seed(42)

age = np.random.randint(25, 60, size=20)          # Tuổi từ 25 đến 60
```

```
    salary = np.random.randint(50000, 150000, size=20)  # Lương từ 50,000 đến 150,000

# Tạo DataFrame để chứa dữ liệu

    data = pd.DataFrame({'Age': age, 'Salary': salary})

# Hiển thị dữ liệu mẫu

    print(data.head())

# Trực quan hóa dữ liệu

    plt.scatter(data['Age'], data['Salary'], color='blue', marker='o', s=50)

    plt.title("Dữ liệu mẫu - Tuổi và Lương")

    plt.xlabel("Tuổi")

    plt.ylabel("Lương")

    plt.show()

# Phân cụm phân cấp tăng dần, sử dụng phương pháp 'ward' để tính khoảng cách

    Z = linkage(data, method='ward')

# Trực quan hóa bằng dendrogram

    plt.figure(figsize=(10, 7))

    dendrogram(Z, labels=data.index.tolist())

    plt.title("Dendrogram của phân cụm phân cấp")

    plt.xlabel("Mẫu")

    plt.ylabel("Khoảng cách")

    plt.show()

# Chọn số cụm bằng cách cắt cây dendrogram tại ngưỡng khoảng cách

    max_distance = 70000  # Ngưỡng khoảng cách (có thể điều chỉnh)
```

```
clusters = fcluster(Z, max_distance, criterion='distance')

# Thêm kết quả phân cụm vào DataFrame

data['Cluster'] = clusters

# Hiển thị dữ liệu sau khi phân cụm

print(data)

# Hiển thị dữ liệu sau phân cụm

plt.scatter(data['Age'], data['Salary'], c=data['Cluster'], cmap='prism', marker='o',
s=50)

plt.title("Kết quả phân cụm phân cấp dựa trên Tuổi và Lương")

plt.xlabel("Tuổi")

plt.ylabel("Lương")

plt.show()
```

3. K_Mean clustering

Một phương pháp phân cụm thuộc học không giám sát phổ biến trong học máy. Mục tiêu của K-Means là chia các điểm dữ liệu thành K cụm sao cho các điểm dữ liệu trong cùng một cụm có khoảng cách ngắn nhất đến tâm cụm của nó.

Cách thức hoạt động của thuật toán K-Means:

- Chọn số lượng cụm K: Xác định số lượng cụm muốn tạo từ dữ liệu.
- Khởi tạo tâm cụm ban đầu: Chọn ngẫu nhiên K điểm từ dữ liệu làm tâm cụm ban đầu (centroid).
- Gán điểm vào cụm: Với mỗi điểm dữ liệu, tính khoảng cách đến từng tâm cụm và gán điểm đó vào cụm có khoảng cách ngắn nhất.
- Cập nhật tâm cụm: Tính toán lại tâm cụm mới cho mỗi cụm bằng cách lấy trung bình các điểm trong cụm.
- Lặp lại: Lặp lại quá trình gán điểm vào cụm và cập nhật tâm cụm cho đến khi:
    - Các tâm cụm không thay đổi nhiều giữa các vòng lặp, hoặc
    - Đạt được số lần lặp tối đa.
- Kết quả cuối cùng: Dữ liệu được phân thành K cụm, mỗi cụm có một tâm cụm đại diện.

Lưu ý: K-Means chỉ tối ưu với dữ liệu dạng số và khoảng cách Euclidean là phổ biến nhất. Tuy nhiên, số cụm *K* phải được xác định trước.

Ví dụ sử dụng thuật toán K-Means để phân cụm dữ liệu 2D.

```
# Thư viện

    import numpy as np

    import matplotlib.pyplot as plt

    from sklearn.cluster import KMeans

    from sklearn.datasets import make_blobs

# Tạo dữ liệu mẫu với 300 điểm và 4 cụm

    X, y_true = make_blobs(n_samples=300, centers=4, cluster_std=0.60,
random_state=0)

# Hiển thị dữ liệu bằng biểu đồ phân tán

    plt.scatter(X[:, 0], X[:, 1], s=50, cmap='viridis')

    plt.title("Dữ liệu mẫu")

    plt.show()

# Khởi tạo và huấn luyện mô hình KMeans với số cụm K = 3

    kmeans = KMeans(n_clusters=3)

    kmeans.fit(X)

# Dự đoán cụm cho mỗi điểm dữ liệu

    y_kmeans = kmeans.predict(X)

# Trực quan hóa kết quả phân cụm

    plt.scatter(X[:, 0], X[:, 1], c=y_kmeans, s=50, cmap='viridis')

# Hiển thị tâm cụm
```

```
centers = kmeans.cluster_centers_

plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75, marker='X')

plt.title("Kết quả phân cụm với K-Means")

plt.show()
```

Xác định số cụm *K* bằng phương pháp Elbow

Phương pháp Elbow giúp xác định số cụm *K* tốt nhất bằng cách xem xét tổng bình phương sai số (Sum of Squared Errors - SSE). SSE giảm khi *K* tăng, nhưng sẽ có một điểm mà SSE giảm chậm hơn nhiều, gọi là "điểm khuỷu tay" (elbow point).

```
# Dùng phương pháp Elbow để xác định K

sse = []

k_range = range(1, 11)

for k in k_range:

    kmeans = KMeans(n_clusters=k)

    kmeans.fit(X)

    sse.append(kmeans.inertia_)  # inertia_ là tổng bình phương khoảng cách của các điểm đến tâm cụm của chúng

# Vẽ biểu đồ Elbow

plt.plot(k_range, sse, marker='o')

plt.xlabel("Số cụm K")

plt.ylabel("Tổng bình phương sai số (SSE)")

plt.title("Phương pháp Elbow để xác định số cụm K")

plt.show()
```

Lưu ý: Nhìn vào biểu đồ Elbow, có thể xác định điểm "khuỷu tay" để chọn số cụm K hợp lý.

II. Bài tập ở lớp

Bài 1. Phân cụm phân cấp với dữ liệu khách hàng

- Giả sử có dữ liệu về khách hàng bao gồm các thuộc tính như tuổi, thu nhập hàng năm và điểm tín dụng.
- Yêu cầu:
    1. Tải dữ liệu từ file CSV (hoặc tự tạo dữ liệu mẫu).
    2. Thực hiện phân cụm phân cấp trên tập dữ liệu dựa trên các thuộc tính này.
    3. Hiển thị dendrogram của quá trình phân cụm.
    4. Chọn số cụm thích hợp dựa trên dendrogram và trực quan hóa kết quả.

Bài 2. Phân cụm phân cấp sinh viên dựa trên điểm số môn học

Mô tả: Một trường học có dữ liệu điểm số của sinh viên trong các môn học như Toán, Vật lý, Hóa học, Sinh học.

Yêu cầu:

1. Phân tích phân cụm để nhóm các sinh viên có kết quả học tập tương tự.
2. Vẽ dendrogram để tìm số lượng nhóm sinh viên.
3. Phân cụm sinh viên dựa trên dendrogram và hiển thị các nhóm.

Bài 3. Phân cụm phân cấp các thành phố dựa trên các chỉ số kinh tế - xã hội

Mô tả : Dữ liệu về các thành phố bao gồm các chỉ số như GDP bình quân đầu người, mức sống, tỷ lệ thất nghiệp và mức độ phát triển giáo dục.

Yêu cầu:

1. Phân cụm các thành phố để nhóm các thành phố có đặc điểm kinh tế - xã hội tương tự nhau.
2. Vẽ dendrogram và xác định số cụm thích hợp.
3. Hiển thị các thành phố thuộc cùng một cụm.

Bài 4. Phân cụm khách hàng dựa trên độ tuổi và chi tiêu với thuật toán K-Mean

Mô tả: giả sử có một tập dữ liệu khách hàng gồm các thông tin về độ tuổi và số tiền chi tiêu.

Yêu cầu:

1. Thực hiện phân cụm K-Means để nhóm các khách hàng thành các cụm dựa trên độ

tuổi và chi tiêu.

2. Vẽ biểu đồ phân tán (scatter plot) để hiển thị các cụm.
3. Thực hiện các giá trị k khác nhau để tìm ra số cụm phù hợp, và sử dụng phương pháp Elbow để chọn số cụm tối ưu.

Bài 5. Phân cụm các sản phẩm dựa trên phản hồi của khách hàng với thuật toán K-Mean

Mô tả bài tập: giả sử có dữ liệu về phản hồi của khách hàng đối với các sản phẩm, bao gồm điểm đánh giá (rating) về chất lượng, giá cả, mẫu mã, độ bền.

Yêu cầu:

1. Phân cụm các sản phẩm dựa trên phản hồi của khách hàng.
2. Đưa ra nhận xét về các cụm, ví dụ: nhóm sản phẩm nào có chất lượng tốt, nhóm nào có giá cao nhưng chất lượng kém.
3. Đề xuất cải tiến cho từng cụm sản phẩm.

III. Bài tập về nhà

Bài 1. Phân cụm khách hàng cho chiến lược tiếp thị

Mô tả: Một doanh nghiệp có dữ liệu khách hàng với các thuộc tính như số lần mua hàng trong tháng, số tiền chi tiêu trung bình mỗi giao dịch, và loại sản phẩm yêu thích.

Yêu cầu:

1. Thực hiện phân cụm khách hàng dựa trên các thuộc tính này để hỗ trợ chiến lược tiếp thị.
2. Xây dựng dendrogram và chọn số cụm phù hợp.
3. Phân tích các cụm để tìm ra những nhóm khách hàng có hành vi mua sắm tương đồng.

Bài 2. Phân cụm văn bản với phân cụm phân cấp

Mô tả: giả sử có một tập dữ liệu gồm nhiều đoạn văn bản hoặc tài liệu (ví dụ: các bài báo).

Yêu cầu:

1. Sử dụng kỹ thuật nhúng văn bản như TF-IDF hoặc Word2Vec để chuyển các đoạn văn bản thành vector số.
2. Thực hiện phân cụm phân cấp trên các vector này.
3. Hiển thị dendrogram của các cụm văn bản.
4. Tạo ra các nhóm văn bản dựa trên dendrogram.

Bài 3. Phân cụm phân cấp trên dữ liệu hình ảnh

Mô tả: giả sử có một tập hợp hình ảnh và mỗi hình ảnh có thể được biểu diễn dưới dạng các đặc trưng số (ví dụ: trung bình màu, độ sắc nét, v.v.).

Yêu cầu:

1. Chuyển mỗi hình ảnh thành vector đặc trưng.
2. Sử dụng phân cụm phân cấp để nhóm các hình ảnh có đặc trưng tương tự.
3. Hiển thị dendrogram và chọn số cụm phù hợp để phân nhóm hình ảnh.
4. Lưu lại kết quả phân cụm cho từng hình ảnh.

Bài 4. Phân cụm dữ liệu iris

Mô tả: Sử dụng tập dữ liệu Iris (trong thư viện sklearn hoặc trên kaggle) với các thuộc tính như độ dài cánh hoa, độ rộng cánh hoa, độ dài đài hoa, độ rộng đài hoa.

Yêu cầu:

1. Áp dụng K-Means để phân cụm các hoa thành các loại khác nhau.
2. So sánh kết quả phân cụm với nhãn loài thực tế để đánh giá độ chính xác.
3. Trực quan hóa kết quả phân cụm và nhãn thực tế.